



Google Summer of Code 2020

**Screwdriver.cd [Continuous
Delivery Foundation]**

Project: Improve SCM Integration

Mentors: Jithin Emmanuel ([@jithine](#)), Tiffany Kyi ([@tkyi](#))

Personal Info

Name: Supratik Das

Email: rick.das08@gmail.com

Telephone: +91 8172003164

Time Zone: India (UTC+5.30)

GitHub: [supra08](#)

University Info

University: Indian Institute of Technology, Roorkee

Major: Electronics and Communication Engineering

Current Year: 3rd year (2021 expected graduation)

Degree: Bachelor of Technology (B.Tech)

Abstract

Screwdriver is a set of services facilitating flawless workflow implementations for continuous delivery pipelines. It provides integrations to interact with various SCMs (Source Control Management Systems) and deliver the project source codes to production.

This project deals primarily to improve the integrations of various SCMs (Source Control Management Systems) in Screwdriver.cd. When developing pipelines for enterprise scale projects, it is imperative that a seamless interface is provided for interacting with private repositories in SCMs. Apart from this, enterprise projects feature pipelines built from multiple dependencies which have active development going on simultaneously. Therefore, it is important to ascertain that the CI/CD tooling works efficiently with these external SCM repositories. Hence, the two key areas where it will be improved are *introduction of deployment keys for seamless handling of private repositories and triggering of builds from external SCM repositories*.

Goals and Deliverables

The following are the deliverables that I wish to achieve during the programme:

- Generation and use of **Deploy Keys** during build pipeline creation.
- **Notification** of SCM changes from **external** repositories.
- Implementation of extensive tests for both.

Implementation Details

Use of Deploy Keys:

Currently, for building a project, Screwdriver provides two possibilities of operation:

- Give the repositories public access and then proceed with the functionalities normally.
- Use SCM tokens (for ex, Personal Access Tokens in case of Github), and add them as **secrets** to the pipeline builds.

These methods work fine until we deal with private repos. The second method is a workaround but it gives the CI/CD tools a user wide access (access to all the repos). In order to make the access limited to the specific repository required for build, Deploy Keys comes into play.

Deploy Key is an **SSH key** specific to a particular repo. The private key rests with the Screwdriver build container and the public key is added to the SCM. The deploy keys are read-only so the CI services cannot make changes to the PRs or commits.

Whenever a new build pipeline is created in SD, it will generate a SSH key pair. The private key thus obtained, is added as secrets in the pipeline builds and the public key as deploy keys in the

respective SCM settings. We can further add an option to add the secret in Pipeline -> options if the user wants to add it later after an initial build.

Prior art: CircleCI, Codeship, Werker

Proposed flow with implementation:

1. The user would run `openssl genrsa -out jwt.pem 2048` and `openssl rsa -in jwt.pem -pubout -out jwt.pub` to generate the public and private keys respectively.
2. The user adds the public key to his preferred SCM. The standard methods for adding the public key as the deploy key in various SCMs are listed below:
 - GH and GHE: [Link 1](#)
 - Bitbucket: [Link 2](#)
 - Gitlab: [Link 3](#)

Alternatively, there will be automatic generation of deploy keys using Screwdriver.

There will be a module `deployKeys.js` in `plugins > pipelines` to manage deploy keys. It's handler would execute the requests `[list, get, add, remove]` to the respective SCMs API endpoint. The various SCM's APIs are listed below:

- GH and GHE: [Link 4](#)
- Bitbucket: [Link 5](#)
- Gitlab: [Link 6](#)

For example, for GH and GHE, we will be using `octokit/rest.js`, and following is the implementation:

```
octokit.repos.addDeployKey({
  owner,
  repo,
  title,
  key,
  read_only
});
```

The parameters are self descriptive, with `read_only` set as true permanently.

3. The private key is now to be added to the pipeline. There are 2 methods for this:
 - **As a secret in the screwdriver.yaml:** In this case the build will start utilizing the deploy key and code checkouts will follow suit (mentioned later).

The config would look somewhat like this:

```

1  shared:
2    image: node:6
3  jobs:
4    main:
5      steps:
6        - print: echo Starting main job!
7      secrets:
8        - DEPLOY_KEY
9      requires:
10       - ~pr
11       - ~commit

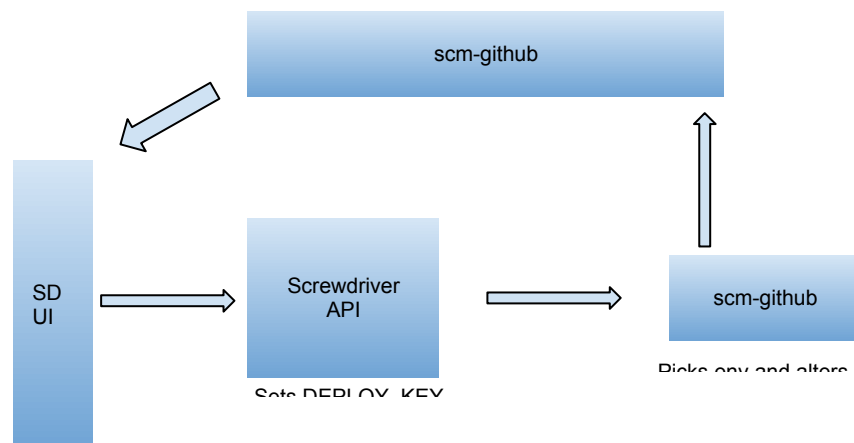
```

Here the `DEPLOY_KEY` is the **private key** generated.

In this case, it is imperative to use a ssh based url as a `checkoutUrl` for the SCM. Thus, if a secret with `DEPLOY_KEY` is present in the config, we will automatically translate any `checkoutUrl` present in HTTPS format to the SSH format and notify the user with the same.

- **From Pipeline > Options in the UI:** A form with an `Add Deploy Key Secret` button in the UI be enough for the purpose. It will execute a request to an endpoint `/addDeployKey` under the pipeline's `deployKeys.js`. Its further flow is as follows:
 - The request, as mentioned, will be handled by the pipeline's `deployKeys.js` which has a wrapper around the `/secrets` endpoint to register a new secret.
 - With the `DEPLOY_KEY` secret in place, the authorization strategy will be modified [more details in **STEP 4**] to utilize the `DEPLOY_KEY`. After this update, the pipeline needs to be rebuilt to reflect the changes.
 - The rebuild can be triggered by sending a PUT request to `/pipeline` with the new `checkoutUrl` as payload. Obviously, the `checkoutUrl` should be a git based. If an https based URI is provided, it will be translated to a git based.

A simplistic flow diagram is given below:



4. With the private key in place as a secret in the pipeline, we now have to configure it to be utilized in code checkouts. Since the secrets are treated as environment variables, we can access them using `process.env.DEPLOY_KEY`.

Octocat provides a number of authentication methods:

<https://octokit.github.io/rest.js/v17#authentication>

Using the AuthApp strategy, the SCM's interface can now be configured to authorize with this:

```
const { Octokit } = require("@octokit/rest");
const { createAppAuth } = require("@octokit/auth-app");

const appOctokit = new Octokit({
  authStrategy: createAppAuth,
  auth: {
    id: 123,
    privateKey: process.env.DEPLOY_KEY
  }
});
```

Some refactoring is required at places where Personal Access Tokens are preconfigured. Henceforth, all git actions of this SCM will be `DEPLOY_KEY` authorized.

Similarly the other SCM's interfaces can be configured as well.

Notifications from external SCM repositories:

For the second goal, we have to implement a functionality to notify a repository (trigger build again) whenever there are source code changes in external dependent repositories. Pipelines can exist whose builds are triggered by changes (releases, tags, commits etc) in external SCM repositories in addition to the repository from which it was created.

Prior Art: Jenkins

The external repos here are termed as **subscribed** and the primary as **subscriber**. A webhook needs to be added to the subscribed repositories in order to receive notifications in the subscriber pipeline.

The external SCM repositories (subscribed) will be listed in the `shared` section of the `screwdriver.yaml` config. The config would look somewhat like this:

```
shared:
  subscribe:
    scmUrls:
      - git@github.com:supra08/external-repo-1.git: [~commit, ~tags, ~release]
      - git@github.com:supra08/external-repo-2.git: [~commit, ~tags, ~release]
```

In order to register hooks from multiple sources the `_createWebhook()` function of the SCM interface needs to be modified. Currently it accepts only the home repo as the target repo, i.e. it accesses the repository for creating webhook using `repo: config.scmlInfo.repo`. This function (`_createWebhook()`) will be reconfigured by iteratively running for all the subscribed repositories from the config and register a webhook with the pipelines' `/webhook` endpoint.

In order to distinguish between subscriber and subscribed repositories, we will add a label tag **alpha** and **beta** respectively. As is evident from the flow, alpha's build will be dependent on betas' and **not** the vice versa.

The `_createWebhook()` function will also test whether the user has permissions to add webhooks to the subscribed repository using a `_hasRepositoryPermissions()` method. It will dry run against the existing keys or Personal Access Tokens available. In case of failure, the user will be instructed on adding his Personal Access Token, or User Keys (discussed later) to the SCM. In this case, Deploy Keys will be avoided since they will not work in this case (discussed later).

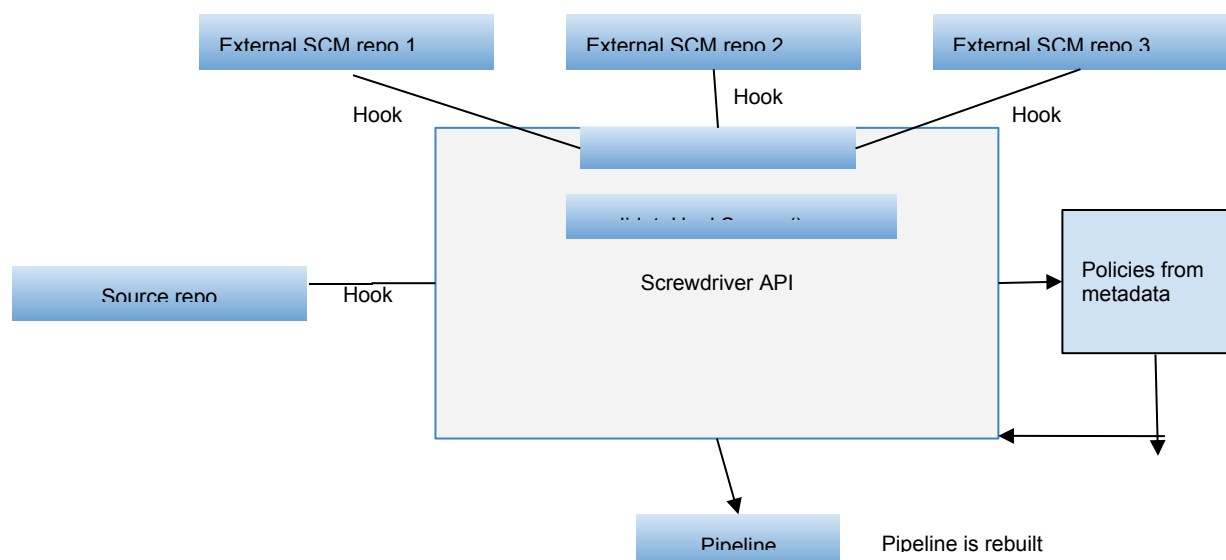
A method `_validateHookSource()` will be implemented in SD's webhooks plugin to validate that the incoming hook notification is actually from the ones listed in the `screwdriver.yaml`. The ones failing will be notified to the user during the build itself, and if the unconfigured source is a required one, the user can add it to the pipeline. For this, an option will be provided in the **Pipeline > Options** in the UI, `Add New Hook Source`. It would be handled from the endpoint `/addNewHookSource` in the webhooks plugin itself utilizing the `_createWebhook()` to add a new hook source.

Since now the metadata will be available from multiple sources, a segregation is required for multiple sources. At this point, **alpha** and **beta** will be labelled to builds from home and external sources. The categorization of the metadata and the subsequent actions (code checkouts in the subscriber pipeline) will be based on test and coverage reports and will be driven by `policies` (discussed in the following section). The metadata will look somewhat like this:

```
meta set meta.status.coverage '{type:"beta","status":"SUCCESS","message":"Coverage is above 80%.",'}
```

As practiced till now, the **alpha** builds will be performed as soon as notification arrives from the webhook. But for **beta** builds, notifications cannot be alone relied upon. In that case, we queue up all the **beta** labelled metadata.

A simplistic flow diagram is given below:



Policies for notifications from external SCM repos:

It will be laid to the user then to select a `policy` in which to roll out builds due to external code changes. The policies will depend on but not limited to the metadata of the subscribed pipelines. Some policies that may be implemented are:

- **Prompt** (on each external build)
- **Tagged** (certain tagged external sources **only** may trigger builds while causing other external changes to be pulled too and applied)
- **Sorted by Coverage** (based on coverage information in the metadata)

As this logic may soon expand, it would be wise to decouple it initially in a separate submodule `sd-external-source-handler` that would handle all the policies and validations of external builds.

User Keys:

Everything works as intended when dealing with public repositories as external sources in pipelines as there are no hindrances to authorization. When we have private repositories, Personal Access Tokens (as is already implemented before) will work fine. **But Deploy Keys will fail to work**, because they are defined as globally unique, i.e. a deploy key can never be used to access multiple repositories. In this case, a new mechanism is proposed, which is almost similar to deploy keys but can be granted access to multiple repositories - User Keys.

Similar to Deploy Keys, User Keys are `user specific SSH` keys. The SCM stores the public key and Screwdriver stores the private key. With the private key, SD has the access comparable to a `git user`. The proposed flow is given below:

1. There will be a module in the pipeline plugin similar to `deployKeys.js`, `userKeys.js`. It will expose endpoints `[list, get, add, remove]` for handling the keys. Alternatively, the UI will provide a similar option.
2. The key pair will be generated with known methods. The public key will be sent to the above endpoint and the private key will be added as a secret `USER_KEY` in the `screwdriver.yaml`.

3. Now since the USER KEY is exposed as an environment variable, the SCM interface (scm-github, scm-bitbucket, scm-gitlab) can pick it up and perform authorization similar to the one provided for deploy keys.
4. An authorization selection handler will be added to the SCM interface that would decide what to use in case both the DEPLOY_KEY and USER_KEY are added in the screwdriver.yaml. Specifically we have two options here, either let the user choose, in which a flag just needs to be set or choose automatically. Identifying the presence of scmUrls in the shared section of the config would refer to multiple external dependencies, and naturally USER_KEY would be preferred.

Proposed Milestones and Timeline

Milestone 1: Implement the Deploy Keys integration necessary documentation and testing.

Milestone 2: Implement the support for external SCM notifications and trigger policies with the documentation and testing.

Milestone 3: Write extensive tests for both the features and perform manual testing considering various corner cases.

Timeline

Timeline	Deliverables
Pre-GSoC	Work on issues and to gain more familiarity with Screwdriver.cd .
Community Bonding (4th May - 1st June)	Continue working on issues and touch some milestone 1 work to create a buffer for testing and debugging as well as achieving the secondary goals. Also a substantial amount of time will be dedicated for bonding with the Screwdriver community.
Phase 1: Deploy Keys integration	
Week 1 - 2 (1st June to 15th June)	Implement the deployKeys.js module in scm-github and replicate the work for scm-gitlab and scm-bitbucket as well. Integrate it with the primary Screwdriver API
Week 3 - 4 (15th June to 29th June)	Continue with any pending implementation. Create the UI interaction and integrate it with the SD API. Add extensive documentation to ease its usage. A substantial time will be spent on testing the modules with test pipelines as well.
Phase 1 Evaluation	

Phase 2: External SCM Notifications

Week 5 - 6
(3rd July to 14th July)

Implementation of User Keys support on the lines of the Deploy Keys. Minor testing phase and modification of the `_createWebhook` method.

Week 7 - 8
(14th July to 27th July)

Implementation of a separate submodule `sd-external-source-handler` and implementation of the policies for triggering builds from external sources. Again some manual testing work.

Phase 2 Evaluation

Phase 3: Tests and manual testing

Week 9 - 10
(31st July to 14th Aug)

Catch up on any pending work from the previous milestone. Start implementation of the testing suite for Deploy Keys integration. This is followed by implementation of the testing suite for External SCM Notifications.

Week 11 - 11.5
(14th Aug to 24th Aug)

Continue with the pending tests if any. Also write the documentation of any pending work. Buffer time for unexpected bugs and also manual testing.

Final Evaluation

About Me

I am a 3rd Year **Bachelor of Technology** student at **Indian Institute of Technology, Roorkee**. I've been involved in software and web development since the past three years, with a number of projects having deployed into production. On campus, I am a part of a group, SDS Labs, whose members are involved in software development in various spheres including but not limited to web development, cloud and infra development and information security.

I've been passionate about programming and software development since my school days, having worked in various personal and group projects. In my freshman year, I was introduced to Docker and Kubernetes. I got a brief stint in my internship as well, working in CI/CD and managing their deployments in the cloud. I am also involved in information security stuff as well, being an active CTF player and conducting lectures and talks.

Related Experience

I've been involved in cloud and infra development for over 1.5 years. I am one of the core developers of a PaaS project **Gaspar** (<https://github.com/sdslabs/gasper>) having fetched over 280 stars. It works on similar lines, packaging the source code from SCMs and deploying them on Docker containers. It features a basic CI pipeline implementation. Every aspect of it is managed by microservices.

Apart from this, I am an open source contributor, having contributed to the Meshery project (<https://github.com/layer5io/meshery>), Linkerd (<https://github.com/linkerd/linkerd2/>) and previously to Owasp Juice Shop (<https://github.com/bkimminich/juice-shop>) and Rocket.Chat (<https://github.com/rocketchat/rocket.chat/>) and also many team projects like Beast (an automated CTF challenge deployment tool), Muzi (A song streaming service in the intranet) and Accounts (Oauth-2 based authorization application, written from scratch).

Apart from this, I have worked in 2 internships where I was given the responsibility of managing the infrastructure, CI/CD and handling deployments in addition to backend development.

Availability

I have my end term examinations scheduled at the end of April, but due to the COVID-19 outbreak, I am uncertain of its exact schedule. So I will be a bit less active during this period. Because of unforeseen events and the pandemic, I may be unavailable for a day or two at maximum, but I will make sure to inform my mentor of such unavailability as early as I can. Apart from this, I have ample time to give the project, an average of about 40 working hours per week. I believe that the allotted work per week is completely doable for me and is neither overloaded nor slacked.

Why me?

I've been quite enthusiastic and eager to learn new things and try my hand on new technologies. I've already worked on a couple of large scale open source projects like Gasper, Linkerd and Meshery and numerous team projects where I've had a stint at CI/CD pipelines and their implementation. I've been developing in Javascript for over 3 years and worked on various tools where asynchronous JS was tapped. Along with that, I've been working in Go for 1.5 years and I believe I can skillfully tackle various stacks. I am confident that I can contribute at par to the standards an open source organization demands.

Apart from this, I would get to work with many professionals as my mentors. This will be a huge learning experience for me. Having worked in numerous team and open source projects, I am aware of the software development process along with the best practices and recommendations.

Involvement after GSoC

I am really interested in working with Screwdriver as a full time contributor because of the experience I'd gather from working in such a large project and the new challenges that I am continually offered. I would be glad to pick up a project from <https://github.com/screwdriver-cd/screwdriver/projects/4> and continue my contributions. Apart from this, I wish to be an active part of the community, gaining and delivering knowledge and actively suggesting and implementing improvements to the project.

References

- <https://documentation.codeship.com/basic/builds-and-configuration/access-to-other-repositories/>
- <https://stackoverflow.com/questions/34757164/jenkins-%E2%87%94-github-webhook-setup-for-multiple-repositories>
- <https://circleci.com/docs/2.0/gh-bb-integration/>